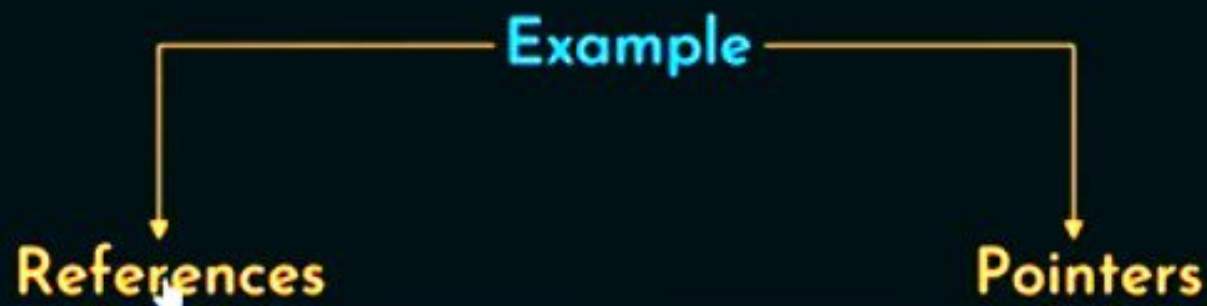


Compound Types

A compound type is a type that is defined in terms of another type.



Simple declarations:

base type

list of variable names

Example:

int

num1, num2;

Generally:

base type

declarators

names a variable and gives the variable a type that is related to the base type.

References

A reference defines an alternative name for an object.

A reference type "refers to" another type.

Defining a reference type: **&d** where **d** is the name being declared

Examples:

```
int ival = 1024;
```

```
int &refVal = ival;    // refVal refers to (is another name for) ival
```

```
int &refVal2;          // error: a reference must be initialized
```

When we define a reference, instead of copying the initializer's value, we **bind** the reference to its initializer.

References

- Once initialized, a reference remains bound to its initial object.
- There is no way to rebind a reference to refer to a different object.
- Because there is no way to rebind a reference, references must be initialized.

A reference is not an object. Instead, a reference is just another name for an already existing object.

After a reference has been defined, all operations on that reference are actually operations on the object to which the reference is bound

Reference Definitions

Multiple references can be defined in a single definition.

Each **identifier that is a reference** must be preceded by the **&** symbol.

Examples:

```
int i = 1024, i2 = 2048;           // i and i2 are both ints
int &r = i, r2 = i2;                // r is a reference bound to i; r2 is an int
int i3 = 1024, &ri = i3;            // i3 is an int; ri is a reference bound to i3
int &r3 = i3, &r4 = i2;              // both r3 and r4 are references
int &refVal4 = 10;                  // error: initializer must be an object
double dval = 3.14;
int &refVal5 = dval;                // error: initializer must be an int object
```

Pointers

A pointer is a compound type that "points to" another type.

They are used for indirect access to other objects.

Differences from References:

- A pointer is an object in its own right.
- Pointers can be assigned and copied → a single pointer can point to several different objects over its lifetime.
- A pointer need not be initialized at the time it is defined. ↓

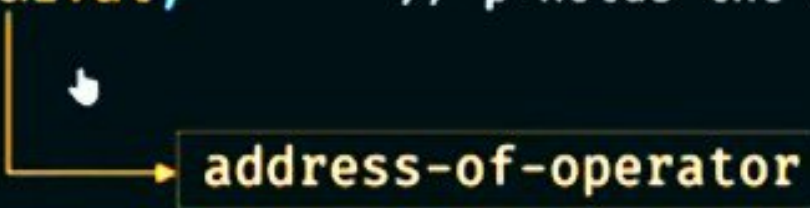
What do Pointers hold ?

A pointer holds the address of another object.

We get the address of an object by using the **address-of operator**
(the **&** operator)

Examples:

```
int ival = 50;  
int *p = &ival;           // p holds the address of ival; p is a pointer to ival
```



address-of-operator

```
cout << p    //gives the address held in pointer p  
cout << *p   // gives the value that is stored in the address pointed by p
```


Examples of Pointer Definitions

Examples:

```
double dval;  
double *pd = &dval;    // ok: initializer is the address of a double  
double *pd2 = pd;      // ok: initializer is a pointer to double  
int *pi = pd;          // error: types of pi and pd differ  
pi = &dval;            // error: assigning the address of a double to a  
                        // pointer to int
```

```
int *p1 = nullptr;  
int *p2 = NULL;  
int *p3 = 0;
```

Null Pointers

A null pointer does not point to any object.

const Qualifier

At times, we may need to define a variable whose value we know cannot be changed.

Example: A variable to refer to the size of a buffer size.

If we use a normal variable → It is easy to change the size if needed.

But if we want to prevent code from inadvertently giving a new value to the variable → It is better to use something that is constant (cannot be changed) ↓

const Qualifier

We can make a variable unchangeable by defining the variable's type as

const:

Example:

```
const int buffSize = 256;           // input buffer size
```



Defined as a **constant**.

Any attempt to assign to **buffSize** is an error.

```
buffSize = 512;           // error: attempt to write to const object ↓
```

Initialization and const

Because we can't change the value of a **const object** after we create it, it must be initialized.

Examples:

```
const int i = get_size();    // ok: initialized at run time
const int j = 23;           // ok: initialized at compile time
const int k;                // error: k is uninitialized const

int a = 21;
const int ci = a;           // ok: the value in a is copied into ci
int b = ci;                 // ok: the value in ci is copied into b
```

const Objects are Local to a File

By default, **const Objects** are local to a file.

Sometimes we have a const variable that we want to share across multiple files but whose initializer is not a constant expression.

To share a const object among multiple files, we must define the variable as **extern**.

const Objects are Local to a File

To share a const object among multiple files, we must define the variable as **extern**.

Example:

// **file_1.cc** defines and initializes a const that is accessible to other files.

```
extern const int buffSize = fcn();
```

// **file_1.h**

```
extern const int buffSize; // same buffSize as defined in file_1.cc
```

References to const

As with any other object, we can bind a reference to an object of a const type.

To do so we use a reference to const.



reference that refers to a const type.

Unlike an ordinary reference, a reference to const cannot be used to change the object to which the reference is bound. 

References to const

Examples:

```
const int ci = 1024;
```

```
const int &r1 = ci;    // ok: both reference and underlying object  
                        are const
```

```
r1 = 42;              // error: r1 is a reference to const
```

```
int &r2 = ci;          // error: non const reference to a const object
```


Initialization and References to const

We can bind a reference to const to a nonconst object, a literal, or a more general expression.

Examples:

```
int i = 42;
```

```
const int &r1 = i;           // we can bind a const int& to a plain  
                             int object
```

```
const int &r2 = 42;          // ok: r2 is a reference to const
```

```
const int &r3 = r1 * 2;      // ok: r3 is a reference to const
```

```
int &r4 = r1 * 2;           // error: r4 is a plain, non const  
  ↑  
reference
```

Initialization and References to const

The same initializations are not legal for nonconst references.

They result in compile-time errors.

But why do they work for const references ?

Let's try to understand this behavior by taking a look at what happens when we bind a reference to an object of a different type.

If we write:

```
double dval = 3.14;  
const int &ri = dval;
```

The compiler transforms this code to

```
int temp = dval;           //create temporary  
                           int from the double  
const int &ri = temp; // bind ri to that  
                           temporary
```

Pointers and const

Like references, we can also define pointers that point to either const or nonconst types.

Like a reference to const, **a pointer to const** may not be used to change the object to which the pointer points.

We may store the address of a const object only in a pointer to const. ↴

Pointers and const

Examples:

```
const double pi = 3.14;           // pi is const; its value may not be
                                   changed

double *ptr = &pi;                 // error: ptr is a plain pointer

const double *cptr = &pi;          // ok: cptr may point to a double that
                                   is const

*cptr = 42;                        // error: cannot assign to *cptr

double dval = 3.14;               // dval is a double; its value can be
                                   changed

cptr = &dval;                      // ok: but can't change dval through cptr
```

const Pointers

Unlike references, **pointers are objects**.

Hence, as with any other object type, we can have a pointer that is itself **const**.

- Like any other **const object**, a **const pointer** must be initialized, and once initialized, its value (i.e., the address that it holds) may not be changed.
- **INDICATION** that the pointer is const —————→ **const after the *.**

→ *This placement indicates that it is the pointer, not the pointed-to type, that is **const**.* ↴

const Pointers

INDICATION that the pointer is const → **const** after the *****.

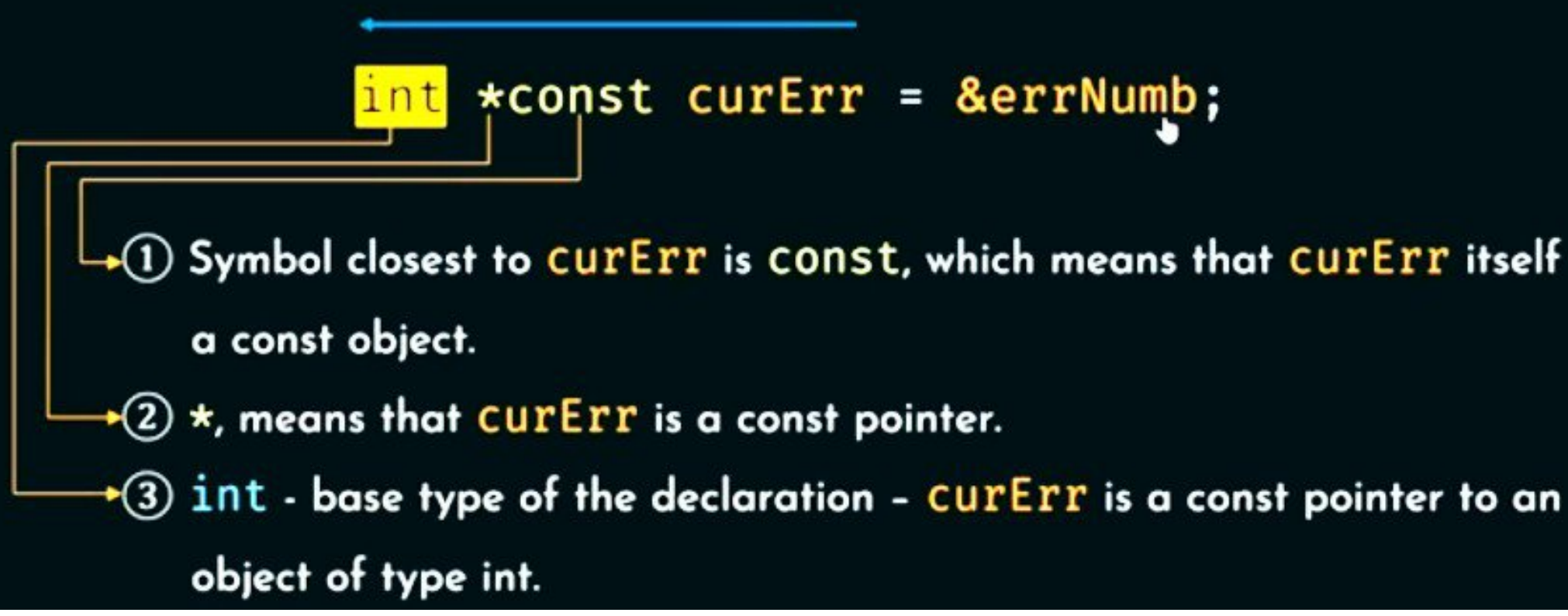
→ *This placement indicates that it is the pointer, not the pointed-to type, that is **const**.*

Examples:

```
int *const curErr = &errNumb;           // curErr will always point to  
                                         errNumb  
  
const double pi = 3.14159;  
const double *const pip = &pi;          // pip is a const pointer to a  
                                         const object
```


const Pointers

Read these declarations from right to left for easy understanding:


`int *const curErr = &errNumb;`

- ① Symbol closest to `curErr` is `const`, which means that `curErr` itself will be a const object.
- ② `*`, means that `curErr` is a const pointer.
- ③ `int` - base type of the declaration - `curErr` is a const pointer to an object of type `int`.

const Pointers

Read these declarations from right to left for easy understanding:

←
`const double *const pip = π`

- ① Symbol closest to `pip` is `const`, which means that `pip` itself will be a const object.
- ② `*`, means that `pip` is a const pointer.
- ③ `const double` - base type of the declaration - `pip` is a const pointer to an object of type const double.

Top-Level and Low-Level const

A pointer is an object that can point to a different object.

So, we can talk independently about

whether a pointer is const and

whether the objects to which it can point are const.

Top-level const:

Indicate that the pointer itself is a const.

Low-level const:

When a pointer can point to a const object, we refer to that const as a low-level const. ↴

Top-Level const

- Indicates that an object itself is const.
- Top-level const can appear in any object type, i.e., one of the built-in arithmetic types, a class type, or a pointer type. ↵

Low-Level const

- Low-level const appears in the base type of compound types such as pointers or references.
- Note that pointer types, unlike most other types, can have both top-level and low-level const independently. ↴

Examples

```
int i = 0;
```

```
int *const p1 = &i;
```

```
// we can't change the value of p1;  
    const is top-level
```

```
const int ci = 42;
```

```
// we cannot change ci; const is top-  
    level
```

```
const int *p2 = &ci;
```

```
// we can change p2; const is low-level
```

```
const int *const p3 = p2;
```

```
// right-most const is top-level, left-  
    most is not
```

```
const int &r = ci;
```

```
// const in reference types is always  
    low-level
```


Copying Objects

The distinction between top-level and low-level matters when we copy an object.

- When we copy an object, top-level consts are ignored.
- On the other hand, low-level const is never ignored.

When we copy an object, both objects must have the same low-level const qualification or there must be a conversion between the types of the two objects.

- In general, we can convert a nonconst to const but not the other way

Copying Objects

Examples:

Top-level consts

```
int i = 0;
```

```
int *const p1 = &i;
```

```
const int ci = 42;
```

```
const int *p2 = &ci;
```

```
const int *const p3 = p2;
```

```
const int &r = ci;
```

```
i = ci;
```

```
// ok: copying the value of ci; top-level const in ci is ignored
```

```
p2 = p3;
```

```
// ok: pointed-to type matches; top-level const in p3 is ignored
```

```
int *p = p3;
```

```
// error: p3 has a low-level const but p doesn't
```

```
p2 = &i;
```

```
// ok: we can convert int* to const int*
```

```
int &r = ci;
```

```
// error: can't bind an ordinary int& to a const int object
```

```
const int &r2 = i;
```

```
// ok: can bind const int& to plain int
```

Constant Expressions

A **constant expression** is an expression whose **value cannot change** and that **can be evaluated at compile time.**

- A literal is a constant expression.
- A const object that is initialized from a constant expression is also a constant expression.

Constant Expressions

A **constant expression** is an expression whose **value cannot change** and that **can be evaluated at compile time.**

Examples:

```
const int max_files = 20;           // max_files is a constant
                                     expression
const int limit = max_files + 1;    // limit is a constant expression
int staff_size = 27;               // staff_size is not a constant
                                     expression
const int sz = get_size();          // sz is not a constant
                                     expression
```

constexpr Variables

Examples:

```
constexpr int mf = 20;           // 20 is a constant expression
constexpr int limit = mf + 1;    // mf + 1 is a constant
                                  // expression
constexpr int sz = size();        // ok only if size is a constexpr
                                  // function
```



Best Practices:

Generally, it is a good idea to use **constexpr** for variables that you intend to use as constant expressions. ↓

Pointers and constexpr

When we define a pointer in a `constexpr` declaration, the `constexpr` specifier applies to the pointer, not the type to which the pointer points.

Examples:

```
const int *p = nullptr;           // p is a pointer to a const int
constexpr int *q = nullptr;       // q is a const pointer to int
```

`p` is a pointer to const, whereas `q` is a constant pointer.

The difference is a consequence of the fact that `constexpr` imposes a top-level const on the objects it defines. 

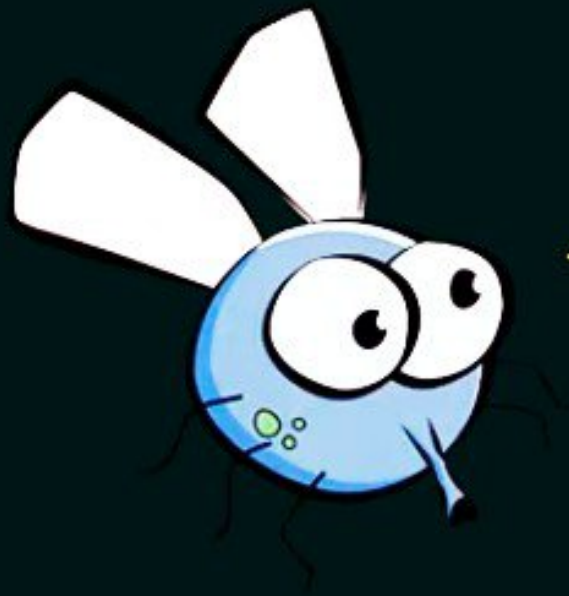
Pointers and constexpr

Like any other constant pointer, a **constexpr** pointer may point to a const or a nonconst type.

Examples:

```
constexpr int *np = nullptr; // np is a constant pointer to int that is null
int j = 0;
constexpr int i = 42;          // type of i is const int
                                // i and j must be defined outside any function
constexpr const int *p = &i; // p is a constant pointer to the const int i
constexpr int *p1 = &j;      // p1 is a constant pointer to the int j
```

Type Aliases



Hi,
My name is
Musca domestica.

But you can call
me **Housefly.**

Type Aliases

A **type alias** is a name that is a **synonym** for another type.

Why to use them?

Simplifies complicated type definitions, making them easier to use.

Lets us emphasize the purpose for which a type is used. ↴

Defining Type Aliases

Method 1: **typedef**

```
typedef double wages;           // wages is a synonym for double
```

```
typedef float percent;         // percent is a synonym for float
```

Defining Type Aliases

Method 2: **using**

```
using char = letter;           // letter is a synonym for char
```

```
using int = num;               // num is a synonym for int
```

Pointers, const, and Type Aliases

```
typedef char *pstring;
```

→ An synonym for char *

```
const pstring cstr = 0;
```

→ A constant pointer to char

```
const pstring *ps;
```

→ A pointer to a constant pointer to char

```
const char *cstr = 0;
```

→ A pointer to const char

Pointers, const, and Type Aliases

```
typedef char *pstring;
```

→ An synonym for char *

```
const pstring cstr = 0;
```

→ A constant pointer to char

```
const pstring *ps;
```

→ A pointer to a constant pointer to char

```
const char *cstr = 0;
```

→ A pointer to const char

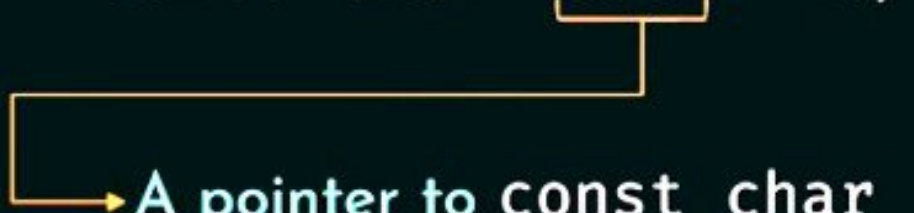
Pointers, const, and Type Aliases

```
const pstring cstr = 0;
```



→ A constant pointer to char

```
const char *cstr = 0;
```



→ A pointer to const char

The base type is char and the * is part of the declarator.

const char is the base type.

This declares cstr as a pointer to const char rather than as a const pointer to char.

The auto Type Specifier

- It's common for us to want to store the **value of an expression** in a **variable**.
- To declare such a variable, we need to know the type of the expression.
- Sometimes, it may be difficult to determine the type of an expression.
- In such cases we can let the compiler figure out the type for us by using the **auto type specifier**.

Since C++11,

The auto Type Specifier

```
some type val1, val2;  
auto sum = val1 + val2;
```

The compiler will deduce the type of `sum` from the type returned by applying `+` to `val1` and `val2`.

If `val1` and `val2` are `int`, `sum` will also be `int`.

If `val1` and `val2` are `double`, `sum` will also be `double`. 

The auto Type Specifier

As a declaration can involve only a single base type, the initializers for all the variables in the declaration must have types that are consistent with each other:

Example:

```
auto i = 0, *p = &i;           // ok: i is int and p is a pointer to int
auto sz = 0, pi = 3.14;       // error: inconsistent types for sz and pi
```

The decltype Type Specifier

- At times, we might want to define a variable with a type that the compiler deduces from an expression but **do not want to use that expression to initialize the variable.**
- For such cases, the new standard introduced a second type specifier, **decltype**, which returns the type of its operand.
- The compiler analyzes the expression to determine its type but does not evaluate the expression.

The decltype Type Specifier

```
decltype(f()) sum = x;    // sum has whatever type f returns
```

- Here, the compiler does not call `f`
- It uses the type that such a call would return as the type for `sum`
- The compiler gives `sum` the same type as the type that would be returned if we were to call `f`.

The decltype Type Specifier

```
const int ci = 0, &cj = ci;
```

```
decltype (ci) x = 0;           // x has type const int
```

```
decltype (cj) y = x;           // y has type const int& and  
                                is bound to x
```

```
decltype (cj) z;               // error: z is a reference and  
                                must be initialized
```


decltype and References

- When we apply **decltype** to an expression that is not a variable, we get the type that that expression yields.
- Some expressions will cause **decltype** to **yield a reference type**.

```
int i = 42, *p = &i, &r = i;
```

```
decltype (r + 0) b;
```

```
// ok: addition yields an int;  
      b is an (uninitialized) int
```

```
decltype (*p) c;
```

```
// error: c is int& and must  
      be initialized
```

decltype and References

decltype of a parenthesized variable is always a reference

- When we apply decltype to a variable without any parentheses, we get the type of that variable.
- If we wrap the variable's name in one or more sets of parentheses, the compiler will evaluate the operand as an expression.

```
decltype ((i)) d;    // error: d is int& and must be initialized
```



decltype and References

decltype of a parenthesized variable is always a reference

- When we apply decltype to a variable without any parentheses, we get the type of that variable.
- If we wrap the variable's name in one or more sets of parentheses, the compiler will evaluate the operand as an expression.

```
decltype ((i)) d;    // error: d is int& and must be initialized
decltype (i) e;      // ok: e is an (uninitialized) int
```


decltype and References

decltype of a parenthesized variable is always a reference

Important

`decltype((variable))` (note, double parentheses) is always
a reference type,

but

`decltype(variable)` is a reference type only if variable is a reference.



Thank you!

Defining and Initializing strings

A string can be initialized in various ways:

<code>string n1</code>	Default initialization; n1 is an empty string
<code>string n2 (n1)</code>	n2 is a copy of n1
<code>string n2 = n1</code>	n2 is a copy of n1
<code>string n3 ("value")</code>	n3 is a copy of the string literal "value"
<code>string n3 = "value"</code>	n3 is a copy of the string literal "value"
<code>string n4 (n, 'c')</code>	Initialize n4 with n copies of the character 'c'

Defining and Initializing strings

Examples:

```
string imempty;  
string helloThere ("How may I help you?") ;  
string theReply = "Can I get some coffee?" ;  
imempty = "Sure! How would you like it?" ;  
string theAnswer = imempty;  
string aLot (10, 's') ;  
string someSs (aLot) ;  
string theRequest ("With a lot of sugar. Thanks") ;
```



string Operations

- Reading and Writing strings
- Reading an Unknown Number of strings
- Using **getline** to read an Entire Line ↴



Reading and Writing strings

```
string readMe;           // empty string
cin >> readMe;          // read a string into readMe
                          // separated by whitespace
cout << readMe << endl; // write readMe to the output
return 0;
```

Reading an Unknown Number of strings

```
string readMe;           // empty string
while (cin >> readMe)    // read until end-of-file
    cout << readMe << endl; // write each string separated by a
                             // new line.

return 0;
```

Using getline to Read an Entire Line

```
string readMe;                // empty string
while (getline(cin, readMe))
// read the input one line at a time until EOF is encountered
cout << readMe << endl;      // write each string separated by a
                               new line.

return 0;
```

string Operations

- The string **empty** operation
- The string **size** operation
- **Comparing** strings

The string empty operation

The **empty function** returns a bool indicating whether the string is empty.

Returns **1** → if the string is **empty**.

Returns **0** → if the string is **not empty**.

We utilize the **dot operator** to define the object on which we wish to execute the empty function when calling this function.



The string empty operation

```
string readMe;  
while (getline(cin, readMe))  
if (!readMe.empty())  
    {  
        cout<<"The string is:"<<endl;  
        cout << readMe << endl;  
    }  
else  
    cout<<"The string is empty" <<endl;
```

The string size operation

The **size function** returns the length of the string.

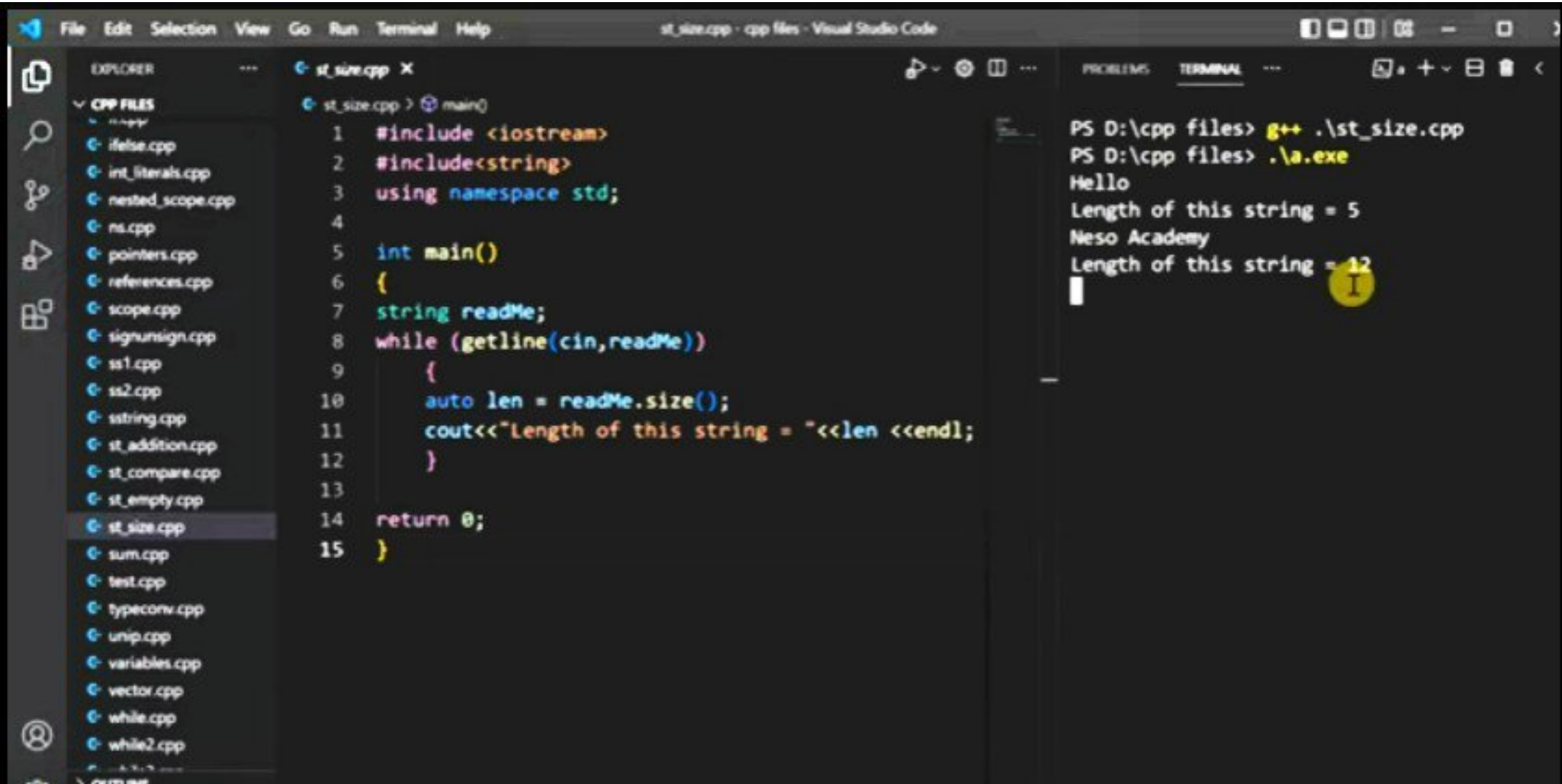
The size function returns a **string::size_type** value.

We utilize the **dot operator** to define the object on which we wish to execute the size function when calling this function. ↴



The string size operation

```
string readMe;  
while (getline(cin, readMe))  
{  
    auto len = readMe.size();  
    cout<<"Length of this string = "<<len <<endl;  
}
```

Comparing strings

Several string comparison operators are defined in the string class.

These operators operate by comparing the strings' characters.

The comparisons are case-sensitive.

Equality Operators:

(==) → Tests the equality of two strings.

(!=) → Tests whether two strings are unequal.



Comparing strings

Relational Operators:

($>$, $>=$, $<$, $<=$) $\rightarrow$ Evaluates whether one string is greater than, greater than or equal to, less than, less than or equal to another.



Comparing strings

1

```
string s1 = "Phone";  
string s2 = "Telephone";  
string s3 = "Telephone Booth";
```

2

```
string s1 = "Book";  
string s2 = "Bookrack";  
string s3 = "Bookkeeper";
```

3

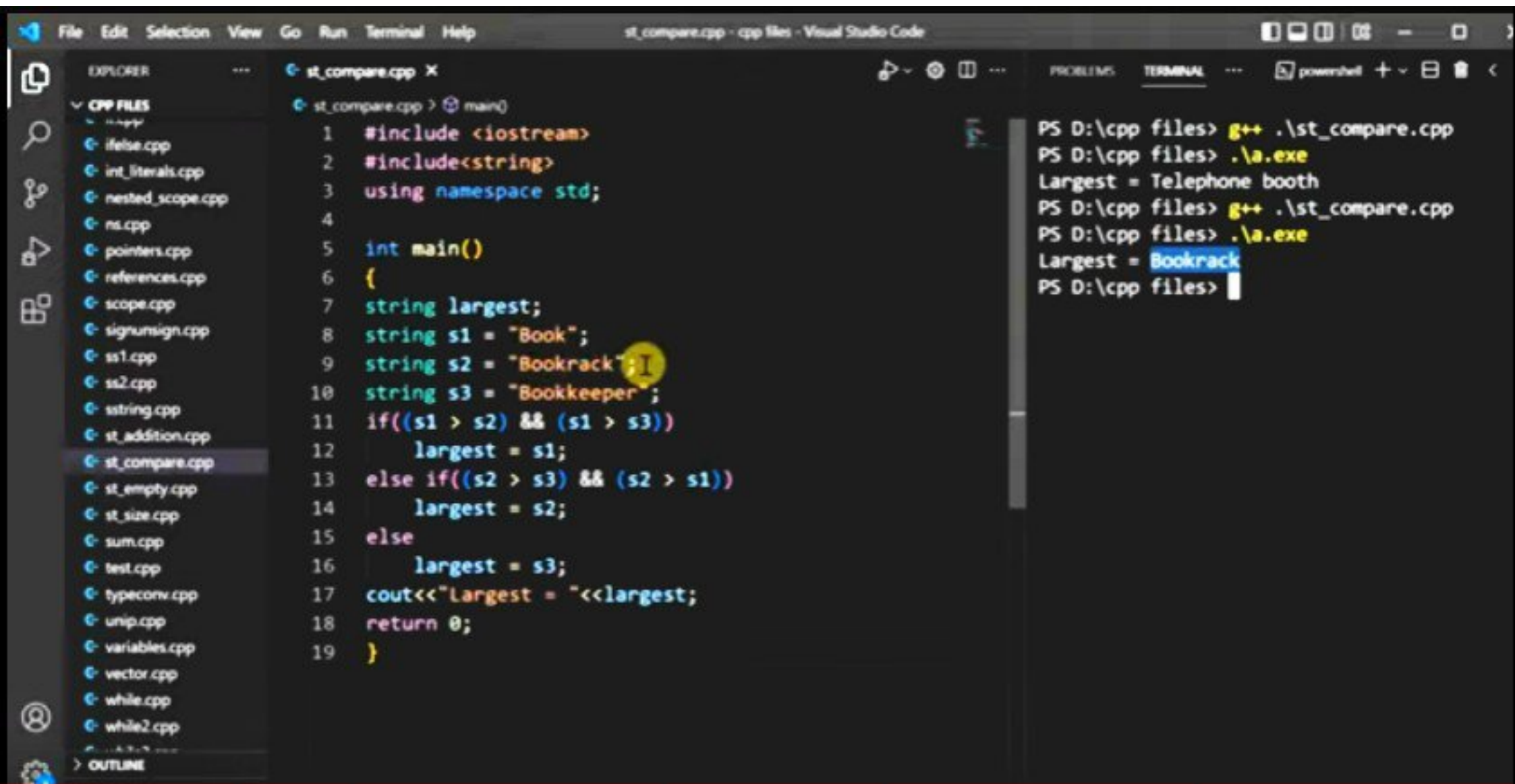
```
string s1 = "Grape";  
string s2 = "Grapefruit";  
string s3 = "Gym";
```


Comparing strings

Relational Operators:

($>$, $>=$, $<$, $<=$) $\rightarrow$ Evaluates whether one string is greater than, greater than or equal to, less than, less than or equal to another.

1. If the lengths of two strings differ and each character in the shorter string is equivalent to its corresponding character in the longer string, the shorter string is shorter than the longer one.
2. When comparing two strings, the first character at which the strings diverge determines the outcome of the string comparison if any characters at matching places in the two strings differ.



Assignment for strings

It is possible to assign:

- One string object to another.
- A literal to a string object.

```
string readMe = "Hello";  
string copyHere = readMe;  
string eS (3, 'E');    // eS is EEE
```



Adding Two strings

A new string is created by concatenating the left-hand operand and the right-hand operand when two strings are added to one another.

```
string lString = "Hello,";  
string rString = "There";  
string readMe = lString + rString; // readMe is Hello,There  
lString += rString; // same as lString + rString
```



Adding Literals and strings

We can concatenate strings with literals but when we mix strings and string or character literals, at least one operand to each + operator must be of string type:

```
string s1 = "Try", s2 = "New";  
string s3 = s1+ "Something" +s3;  
string s4 = s3+ "Everyday";  
string s5 = "Try" + "Something"; // error: no string operand
```

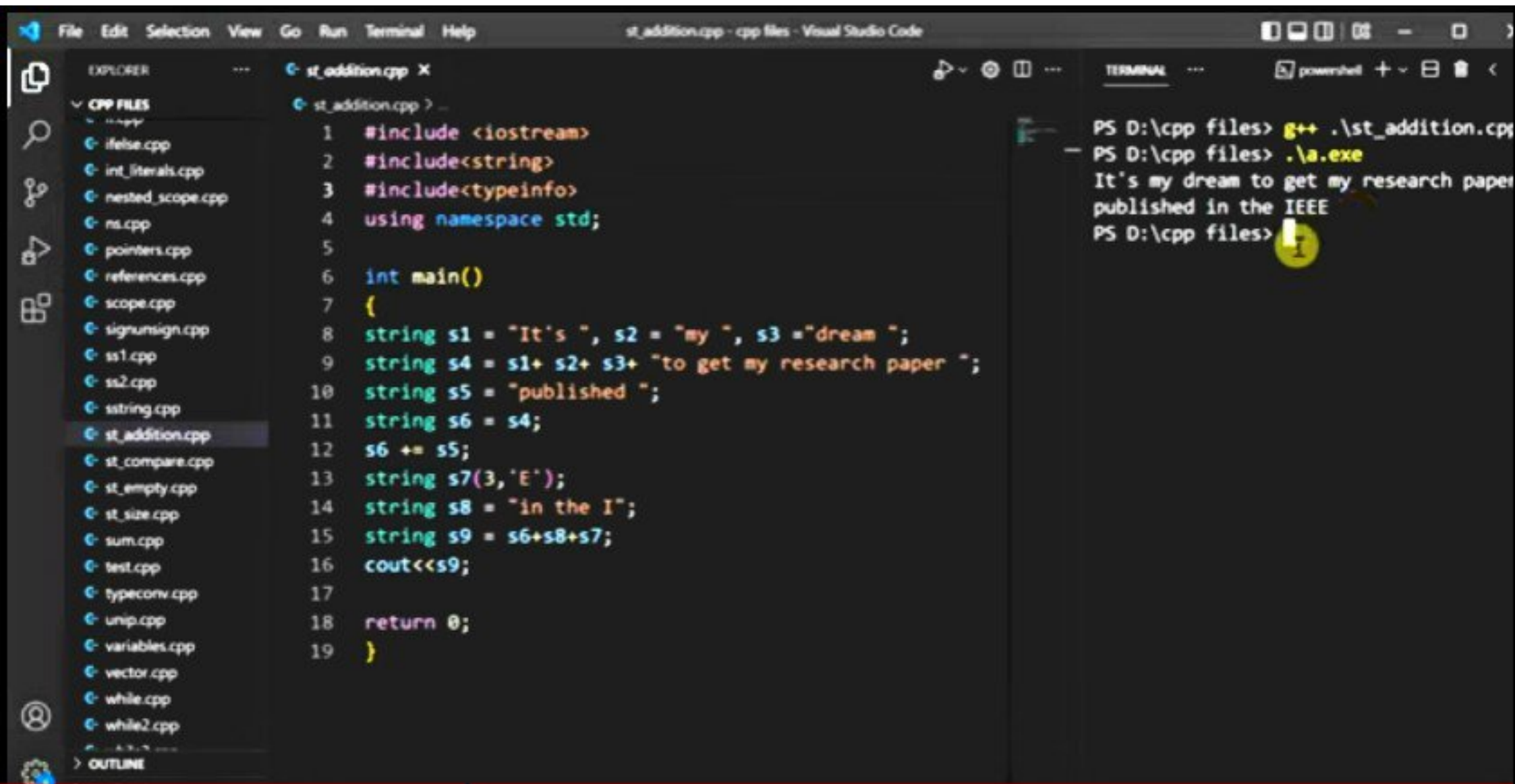
string +
"Abc"
+ string

Adding Literals and strings

We can concatenate strings with literals but when we mix strings and string or character literals, at least one operand to each + operator must be of string type:

```
string s1 = "Try", s2 = "New";  
string s3 = s1+ "Something" +s3;  
string s4 = s3+ "Everyday";  
string s5 = "Try" + "Something"; // error: no string operand  
string s6 = "Try" + "Something" +s2; // error: can't add string
```

string +
"Abc"
+ string



Dealing with Characters in a string

While dealing with string there may be cases when we might want to:

- Check whether a string contains any whitespace.
- Change the characters to lowercase/ uppercase.
- See whether a given character is present, etc. 

Functions used for processing characters

`isalnum (c)`

True if `c` is a letter or a digit

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>isctrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

A b c
1 2 3

Functions used for processing characters

<code>isalpha (c)</code>	True if c is a letter.
--------------------------	------------------------

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>isctrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

A B C
a b c

Functions used for processing characters

`isctrl (c)`

True if `c` is a control character.



<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>isctrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

lt ln lb
lr

Functions used for processing characters

<code>isdigit (c)</code>	True if c is a digit.
--------------------------	-----------------------

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>isctrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

1 2 3
4 5

Functions used for processing characters

`isgraph (c)`

True if `c` is not a space but is printable.

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>iscntrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

A
a
1

Functions used for processing characters

`islower (c)`

True if `c` is a lowercase letter.



<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>isctrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

a b c
d e

Functions used for processing characters

`isupper (c)`

True if `c` is an uppercase letter.

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>isctrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

A B C
D E

Functions used for processing characters

`isprint (c)`

True if `c` is a printable character.

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>iscntrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

A b c
1 2

Functions used for processing characters

`ispunct (c)`

True if `c` is a punctuation character.

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>isctrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

! , .
\$ @

Functions used for processing characters

`isspace (c)`

True if `c` is a whitespace.

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>isctrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

`'f'`
`'n'` `'t'`

Functions used for processing characters

<code>isxdigit (c)</code>	True if c is a hexadecimal digit.
---------------------------	-----------------------------------

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>isctrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

A B C
D E F

Functions used for processing characters

`tolower (c)`



Change `c` to lowercase if it is an uppercase letter. Otherwise leaves it unchanged.

<code>isalnum (c)</code>	<code>isalpha (c)</code>	<code>iscntrl (c)</code>
<code>isdigit (c)</code>	<code>isgraph (c)</code>	<code>islower (c)</code>
<code>isupper (c)</code>	<code>isprint (c)</code>	<code>ispunct (c)</code>
<code>isspace (c)</code>	<code>isxdigit (c)</code>	<code>tolower (c)</code>
<code>toupper (c)</code>		

A → **a**
B → **b**
c → **c**

Thank you!

Processing Every Character in a String

Use the `range` `for` statement to iterate through the elements in a given sequence and performs some operation on each value in that sequence.

Example – 1

Print each character from a string on a new line:

```
string readMe ("Hello There");  
for (auto ch : readMe)  
    cout << ch << endl;
```

Example - 2

Count the number of punctuation characters in a string:

```
string readMe ("Hello, There!!!");  
int punct_count = 0;  
for (auto ch : readMe)  
    if (ispunct(ch))  
        ++punct_count;  
cout << "Number of punctuation characters in "<<readMe  
    <<" = " <<punct_count;
```


Example – 3

Convert all characters of a string to uppercase:

```
string readMe ("Hello, There!!!");
```

```
for (auto &ch : readMe)
```



```
    ch = toupper (ch);
```

```
cout << "Converted string = " << readMe;
```

Example - 3

Convert all characters of a string to uppercase:

```
string readMe ("Hello, There!!!");
```

```
for (auto &ch : readMe)
```

ch is a reference

```
    ch = toupper (ch);
```

Changes the character in readMe

```
cout << "Converted string = "<<readMe;
```

Processing Some Characters in a String

Sometimes we simply need to access one character, or we only need to access characters up until a certain condition is met.

Example:

Capitalize just the first character in a string:

hello → Hello

Capitalize just the first word in a string:

hello there → HELLO there



Using a SUBSCRIPT to access individual characters in a string

The **subscript** operator (the `[]` operator) denotes the position of the character we want to access.

It returns a reference to the character at the given position.

Subscript for a string:

Starts at `0`

Ends at `size() - 1`

`0 1 2 3 4`
`↑ ↑ ↑ ↑ ↑`
`H E L L O` Size = 5

The values we use to subscript a string must be `>= 0` and `< size()`

Example – 1

Print only the first character of a string:

```
string readMe ("Hello There");  
if (!readMe.empty())           // make sure that string is not empty  
    cout << readMe[0] << endl; // print the first character in readMe
```

Example - 2

Capitalize the first letter of a string:

```
string readMe ("hello there");  
if (!readMe.empty())  
    readMe[0] = toupper(readMe[0]);  
cout<< readMe <<endl;
```

Example – 3

Capitalize the entire first word of a string:

```
string readMe ("hello there");  
decltype(readMe.size()) i;  
for (i = 0; i != readMe.size() && !isspace(readMe[i]); ++i)  
    readMe[i] = toupper(readMe[i]);  
cout<< readMe <<endl;
```

Example – 4

[Random Access] Generate a hexadecimal representation of a given valid input:

```
const string hexdigits = "0123456789ABCDEF"; // possible hex digits
cout << "Enter a string of 0-15 numbers, separated by spaces. When
finished, press ENTER: "<<endl;
string result; // will hold the resulting hexadecimal string
string::size_type ip; // hold numbers from the input
while (cin >> ip)
if (ip < hexdigits.size()) // ignore invalid input
result += hexdigits[ip]; // fetch the indicated hex digit
```


File Edit Selection View Go Run Terminal Help

sts_ex4.cpp - cpp files - Visual Studio Code

08

EXPLORER

CPP FILES

ss2.cpp

sstring.cpp

st_addition.cpp

st_compare.cpp

st_empty.cpp

st_ex1.cpp

st_ex2.cpp

st_ex3.cpp

st_size.cpp

sts_ex1.cpp

sts_ex2.cpp

sts_ex3.cpp

sts_ex4.cpp

sum.cpp

test.cpp

typeconv.cpp

unip.cpp

variables.cpp

vec1.cpp

vector.cpp

while.cpp

while2.cpp

while3.cpp

OUTLINE

sts_ex4.cpp > main()

1 #include <iostream>

2 #include<string>

3 using namespace std;

4

5 int main()

6 {

7 const string hexdigits = "0123456789ABCDEF";

8 cout << "Enter a string of 0-15 numbers,"

9 <<"separated by spaces."

10 <<"When finished, press ENTER: "<<endl;

11 string result;

12 string::size_type ip;

13 while (cin >> ip)

14 if (ip < hexdigits.size())

15 result += hexdigits[ip];

16 cout << "Your hex number is: " << result << endl;

17 return 0;

18 }

TERMINAL

powershell

PS D:\cpp files> g++ .\sts_ex4.cpp

PS D:\cpp files> .\a.exe

Enter a string of 0-15 numbers,separated by spaces.When finished, press ENTER:

12 10 15 14

^Z

Your hex number is: CAFE

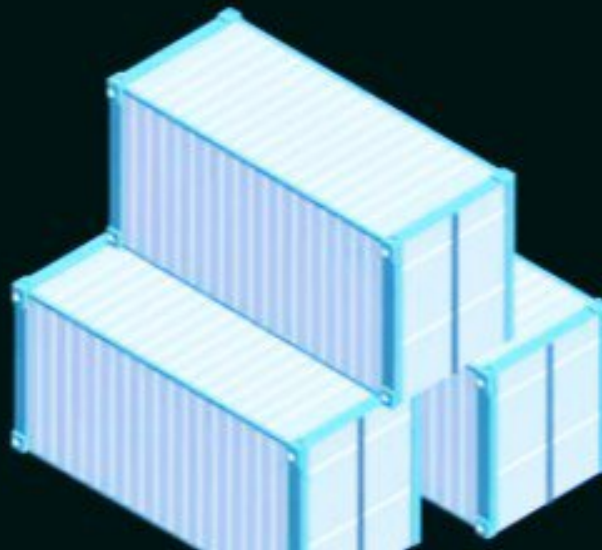
PS D:\cpp files>

The Vector Type

Vectors are sequence **containers** that represent **variable-size arrays**.

↓
contains other objects

↓
collection of objects



Using Vectors

Include the appropriate header:

```
#include <vector>  
using std::vector;
```

Class Template

A vector is a **class template**.



A blueprint from which specific class types can be created.

Template:

- Not functions or classes
- Guidelines for the compiler to use to create classes or functions.

Instantiation → Describes the procedure the compiler employs to build classes or functions out of templates.

Class Template

We specify which class to instantiate by supplying additional information, depending on the template.

HOW ?

Enclose it in two angle brackets after the template's name.

For Vectors:

We specify the type of the objects the vector will hold

```
vector<int> myVec
```



Example

```
#include <iostream>
#include<vector>
using namespace std;

int main()
{
    vector<int> myVec;
    myVec = {1,2,3,4,5};
    for(int i:myVec)
        cout<<i<<endl;

    return 0;
}
```